

Ecolingo — Guía del Proyecto

Todo lo que no es código de una feature: repositorio, diagramas del TCC, y despliegue/monitoreo del piloto.

Campo	Valor
Proyecto	Ecolingo — Plataforma Web Gamificada para Educación Financiera
Institución	Universidad Columbia del Paraguay — Ingeniería Informática
Autores	Cesar Luis Alcaraz Casco · Martín Vega
Tutor	Prof. José Alfredo Souberlich Román
Versión del documento	1.0
Fecha	Agosto 2026
Reemplaza a	01-PRE-CODIGO.md, 03-DESPLIEGUE-Y-MONITOREO.md
Compañero	GUIA_DE_CODIGO.md — ahí vive todo lo que sí es código de features

Cómo se usa este documento

Tiene dos partes, separadas por un divisor claro:

- **Parte A — Antes del código.** Cuentas de servicio, repositorio, y los diagramas que pide el TCC (arquitectura, esquema de datos, casos de uso). Se hace una sola vez, al principio.
- **Parte B — Después del código.** Despliegue a producción, monitoreo del piloto, incidentes, cierre. No se toca hasta que las features del núcleo de `GUIA_DE_CODIGO.md` estén hechas y probadas.

No hay `00-INDICE.md` en este sistema de 3 archivos. Si necesitan saber en qué está el proyecto, alcanza con revisar los checklists de este documento y del otro — ambos son la fuente de verdad de su propia parte.

PARTE A — Antes del código

Bloque 1 — Cuentas de servicio

Responsable: repartido. **Duración:** 30 min.

Servicio	Para qué	Crea	Plan
GitHub	Repositorio y revisión de PR	(ya existe)	Free
MongoDB Atlas	Base de datos	A	Free
Render	Backend	B	Free

Servicio	Para qué	Crea	Plan
Vercel	Frontend	B	Hobby

Regla innegociable

Ambos tienen que poder administrar las cuatro cuentas desde el primer día. No "después". No "cuando haga falta".

El escenario que esto previene es concreto: octubre, mitad del piloto, la persona que creó la cuenta de Render pierde el acceso o no está disponible, y no se puede tocar el backend. Cuatro meses de trabajo dependiendo de que una persona conteste el teléfono.

Cómo se logra cambia según el servicio — dos de los cuatro ya no ofrecen invitación gratuita.

Servicio	Cómo	Costo
Atlas	Access Manager del proyecto → invitar por email → rol Project Owner	Gratis
GitHub	Settings del repo → Collaborators → permiso Admin	Gratis
Render	El workspace gratuito (Hobby) no permite invitar miembros — recién se habilita en el plan Pro (USD 25/mes)	Pago
Vercel	El plan gratuito (Hobby) tampoco permite invitar colaboradores — recién en Pro (USD 20/mes por miembro)	Pago

Verificado en agosto 2026. Tanto Render como Vercel restringieron la colaboración multiusuario a sus planes pagos. No es un error de configuración si el botón de invitar pide upgradear el plan — es la política actual de ambos.

Para Render y Vercel, en lugar de invitar: compartir el login de la cuenta. Pagar ~USD 45/mes combinados solo para tener dos usuarios administrativos no tiene sentido para un piloto de cuatro semanas con presupuesto USD 0. La cuenta se crea una vez (a nombre de quien la cree, Persona B según el reparto de arriba) y el email + contraseña de esa cuenta se agregan como **un secreto más** al canal privado de credenciales de la sección siguiente. Es el mismo compromiso que ya asumen con `JWT_SECRET` o la contraseña de `ecolingo_app`.

Riesgo de esto: si uno de los dos cambia la contraseña sin avisar, el otro queda afuera. Acuerden explícitamente no rotarla sin decírselo al otro primero — y sí rotarla al terminar el piloto (Bloque 9.2 de la Parte B).

Credenciales compartidas

Van a acumular secretos: login de Render, login de Vercel, contraseña de `ecolingo_app`, `JWT_SECRET`, `FORMS_INTEGRATION_KEY`, contraseña del docente. Acuerden **un solo canal privado** para todos ellos y úsenlo siempre.

Sirve un gestor de contraseñas compartido, un documento privado en Drive, o un chat directo entre ustedes dos. Lo que no sirve: mensajes sueltos en distintos lugares, ni —bajo ninguna circunstancia— el repositorio.

✓ Checkpoint

- ☐ Cuatro servicios con cuenta activa
- ☐ Ambos con acceso real de administrador en Atlas y GitHub (invitación) y en Render y Vercel (login compartido)
- ☐ Canal privado de credenciales acordado, incluyendo los logins de Render y Vercel

Bloque 2 — Repositorio

Responsable: Persona B ejecuta, Persona A revisa. **Duración:** 45 min.

Se crea de cero, ya estructurado como monorepo desde el primer commit — más simple que reorganizar el scaffold que había, que no tenía historial que valiera la pena conservar.

2.1 Estructura destino

```
ecolingo/
├── .github/
│   ├── ISSUE_TEMPLATE/feature.md
│   └── pull_request_template.md
├── backend/           ← se llena siguiendo GUIA_DE_CODIGO.md, §0.7
├── frontend/         ← se crea con `npm create vite`, acá abajo
├── docs/             ← los 3 documentos del proyecto
├── .gitignore
└── README.md
```

2.2 Crear el repositorio desde cero

En VS Code, con la terminal integrada (Ctrl+` o Cmd+`):

```
# 1. Carpeta local vacía y control de versiones
mkdir ecolingo && cd ecolingo
git init
code . # si no se abrió sola, abre esta carpeta en VS Code

# 2. Scaffold del frontend – genera frontend/ con Vite + React
npm create vite@latest frontend -- --template react

# 3. El resto de la estructura
mkdir -p backend docs .github/ISSUE_TEMPLATE
```

Copió los 3 documentos (Tesis, GUIA_DE_CODIGO.md, GUIA_DEL_PROYECTO.md) a docs/ — arrastrándolos en el explorador de VS Code alcanza.

Recién ahora se crea el repositorio en GitHub, vacío, sin README ni .gitignore ni licencia — todo eso ya lo vas a tener local en el primer commit:

```
# 4. Conectar con GitHub (creá el repo vacío ahí primero, copiá su URL)
git remote add origin https://github.com/<usuario>/ecolingo.git
git branch -M main
git add -A
git commit -m "chore: estructura inicial del monorepo"
git push -u origin main

# 5. Rama de integración
git checkout -b dev
git push -u origin dev
```

Con esto ya está lo que hacía `git mv` en la versión anterior de esta guía, pero sin el paso intermedio: no hay nada que mover porque la estructura nace correcta. Invitá al otro como administrador del repo (Bloque 1) antes de seguir.

2.3 .gitignore de la raíz

```
# Dependencias
node_modules/

# Entorno
.env
.env.local
.env.*.local

# Builds
dist/
build/
.vercel/

# RespalDOS de la base – contienen datos de alumnos
backups/

# Sistema y editor
.DS_Store
Thumbs.db
*.log
.vscode/*
!.vscode/extensions.json
!.vscode/settings.json
```

`backups/` está ahí desde el primer día por una razón: los respaldos de producción contienen nombres y códigos de estudiantes menores de edad. La tesis §4.5 compromete cumplimiento de la Ley N.º 6.529/2020 de Protección de Datos Personales. Un respaldo en un repositorio público sería un incumplimiento directo.

2.4 Ramas

Tres niveles:

Rama	Rol	Se despliega en
<code>main</code>	Producción. Solo código verificado.	Render prod + Vercel prod
<code>dev</code>	Integración. Todas las features se mergean acá primero.	Render dev (<i>opcional</i>) + previews de Vercel
<code>feat/FXX-nombre</code>	Trabajo individual. Una por feature.	Preview automática de Vercel

`main` y `dev` ya existen desde el paso 2.2. De acá en más, toda feature sale de una rama `feat/FXX-nombre` creada desde `dev`.

2.5 Proteger las ramas

En **Settings** → **Branches** del repositorio, una regla para `main` y otra para `dev`:

Opción	<code>main</code>	<code>dev</code>
Require a pull request before merging	✓	✓
Require approvals	1	1
Require conversation resolution	✓	✓
Do not allow bypassing	✓	—
Allow force pushes	✗	✗
Allow deletions	✗	✗

Con dos personas, exigir una aprobación significa que **nadie mergea su propio código**. Es exactamente el principio de revisión cruzada del brief, aplicado por la herramienta en vez de por la memoria. También garantiza que ambos hayan leído todo el código del proyecto — que es el requisito para defenderlo entero ante el tribunal.

2.6 Plantilla de Pull Request

`.github/pull_request_template.md`

```
## Feature
<!-- F03 – Mapa de Módulos -->

## Qué hace este PR

## Endpoints tocados
<!-- Números de GUIA_DE_CODIGO.md, Parte 1. "Ninguno" si no aplica. -->

## Checklist
- [ ] Los endpoints responden con el shape exacto de GUIA_DE_CODIGO.md,
```

```

Parte 1
- [ ] Los errores usan el envoltorio estándar y códigos del catálogo (§0.6)
- [ ] La pantalla maneja carga, error y vacío
- [ ] Responsivo en 375 px, 768 px y 1280 px
- [ ] Colores desde los tokens de Tailwind, sin hexadecimales literales
- [ ] Sin secretos ni `.env` en el diff

## Documentación
- [ ] No modifica ningún documento
- [ ] Modifica documentos → ⚠️ **re-subir al contexto de Claude:**
  _(cuáles)_

## Capturas
<!-- Obligatorio si hay cambios de UI -->

```

2.7 Plantilla de Issue

.github/ISSUE_TEMPLATE/feature.md

```

---
name: Feature
about: Trabajo sobre una feature de GUIA_DE_CODIGO.md
title: '[FXX] '
labels: feature
---

## Feature
<!-- ID y nombre según GUIA_DE_CODIGO.md, Parte 0.13 (mapa de features) -->

## Alcance de este issue

## Criterios de aceptación
<!-- Copiados de GUIA_DE_CODIGO.md, Parte 3 -->
- [ ]
- [ ]

## Depende de
<!-- Otras features o tareas que deben estar listas antes -->

```

2.8 Etiquetas

En **Issues** → **Labels**, creá: **F01** a **F12**, más **backend**, **frontend**, **bug**, **docs**, **bloqueado**.

Con **bloqueado** alcanza para ver de un vistazo si alguien está esperando algo del otro — que es la información más útil cuando trabajan en paralelo.

2.9 Actualizar la documentación más adelante

Los 3 documentos ya quedaron en `docs/` desde el commit inicial del paso 2.2. Si más adelante cambia alguno (nueva versión de `GUIA_DE_CODIGO.md`, por ejemplo), el flujo es:

```
git checkout dev && git checkout -b docs/actualizar-guia-codigo
# Reemplazar el archivo correspondiente en docs/
git add docs/
git commit -m "docs: actualizar GUIA_DE_CODIGO.md a v2.1"
git push origin docs/actualizar-guia-codigo
# Abrir PR contra dev, que lo revise el otro, mergear.
```

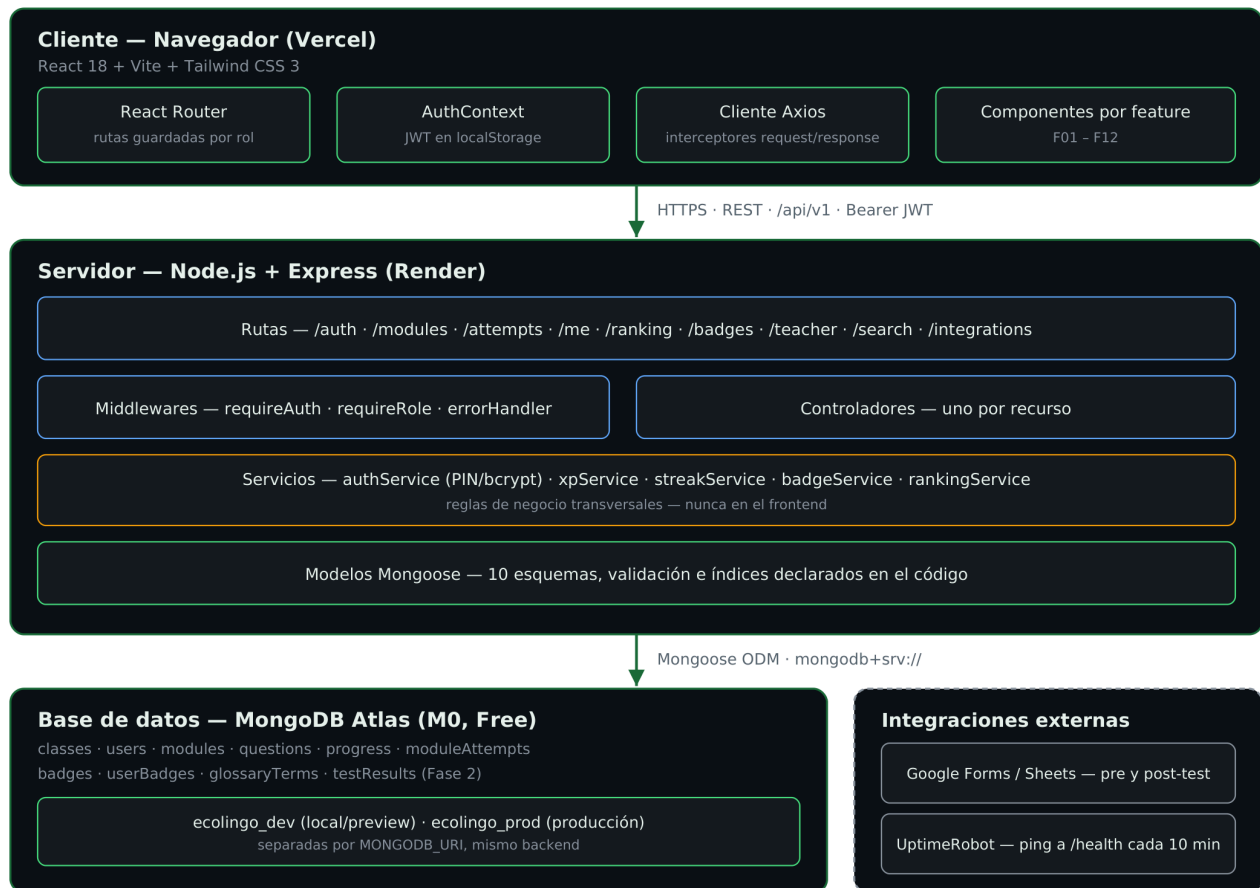
Y **re-subir el archivo actualizado al contexto de Claude de ambos** — es la misma regla que ya está en la plantilla de PR (§2.6).

✅ Checkpoint

- ☐ Monorepo con `frontend/`, `backend/`, `docs/`
- ☐ `main` y `dev` existen y están protegidas
- ☐ Plantillas de PR e Issue en `.github/`
- ☐ Etiquetas creadas
- ☐ Documentación en `docs/`
- ☐ `git status` no lista `.env` ni `node_modules`

A.3 Diagrama de Arquitectura de Software

Diagrama de Arquitectura de Software — Ecolingo



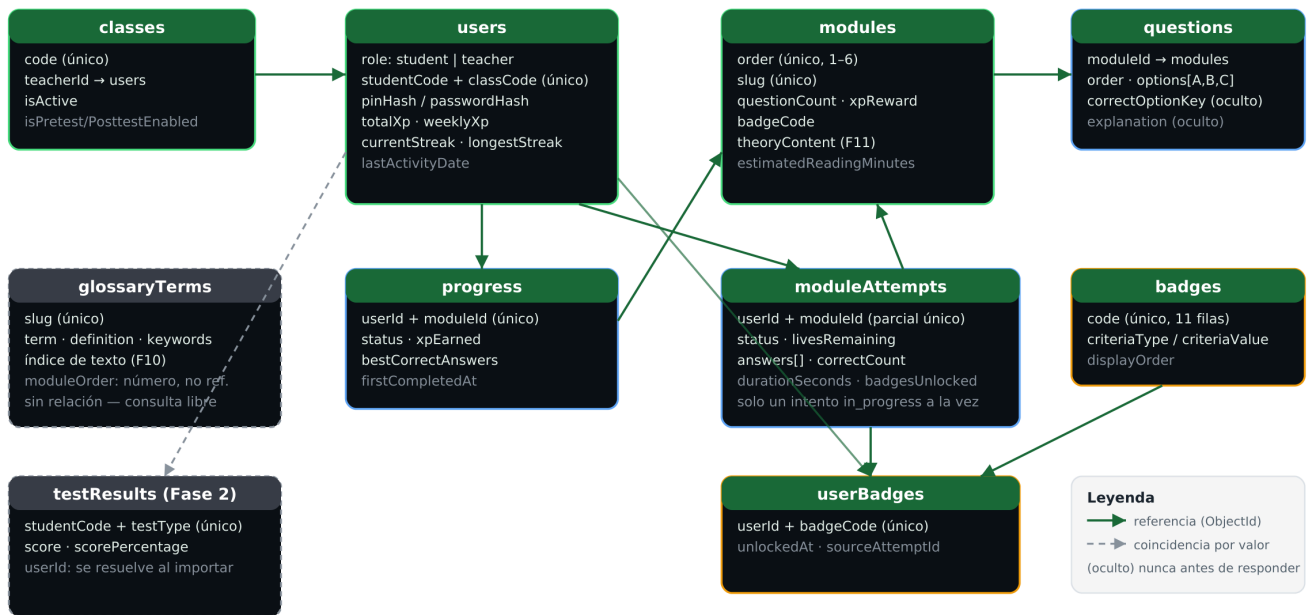
Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026

Tres capas — cliente (Vercel), servidor (Render), base de datos (MongoDB Atlas) — más las dos integraciones externas (Google Forms/Sheets, UptimeRobot). El detalle exacto de cada bloque del servidor (rutas, middlewares, servicios, modelos) está desarrollado feature por feature en [GUIA_DE_CODIGO.md](#), Partes 1 y 2. Este diagrama es la vista de conjunto que pide el TCC — insertarlo en el Capítulo 4 de la tesis, sección de arquitectura.

A.4 Esquema de colecciones de MongoDB

Esquema de colecciones — MongoDB (Ecolingo)

Relaciones lógicas por referencia (ObjectId). Sin llaves foráneas: MongoDB no las aplica — la integridad la garantizan los índices y la capa de servicio.

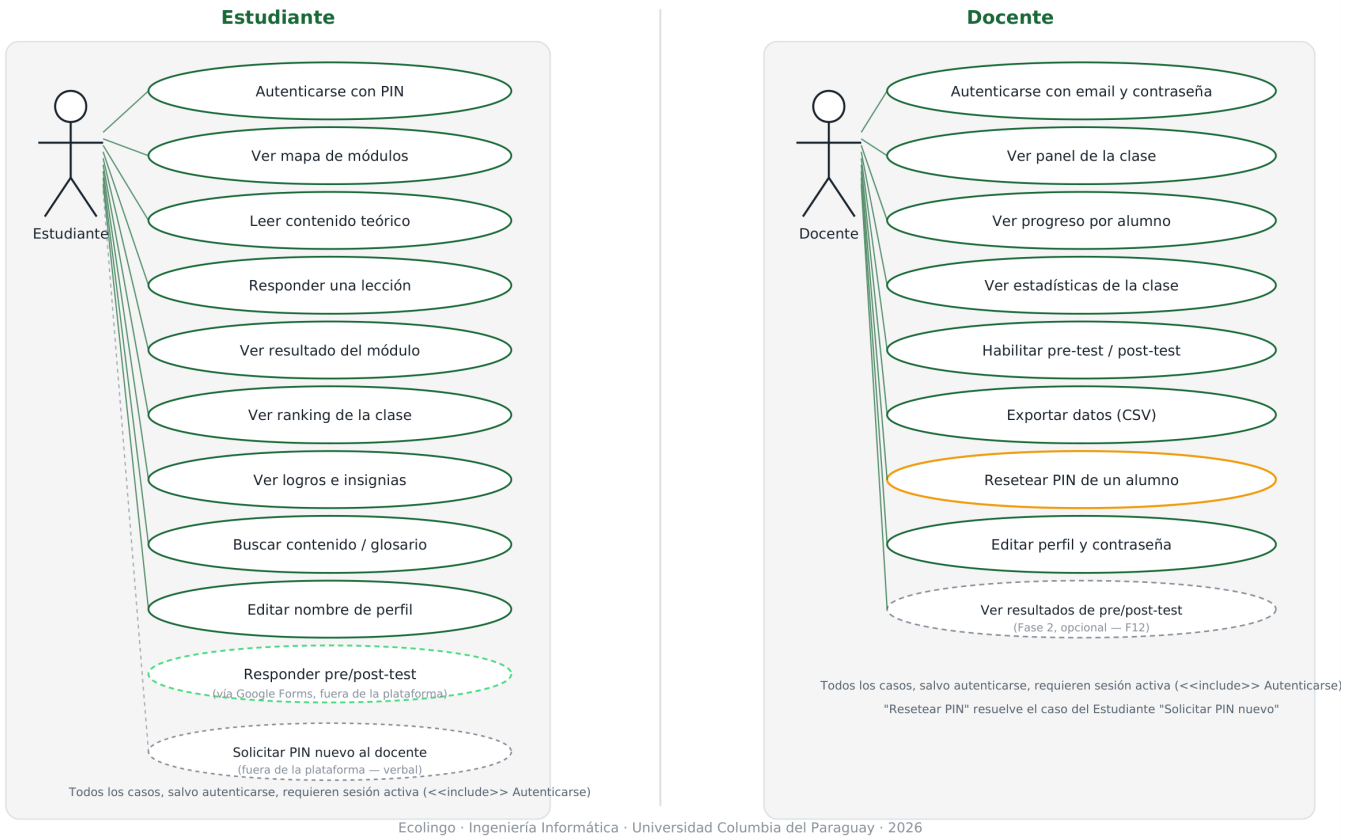


Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026

Es el equivalente de un diagrama ER para una base de datos sin llaves foráneas: los óvalos verdes son referencias reales por **ObjectId** (las que Mongoose puede poblar con `.populate()`), y la línea punteada gris es una coincidencia por valor (**studentCode**) sin relación declarada en el esquema — la conciliación la hace la capa de servicio, no la base. El detalle campo por campo de cada colección está en [GUIA_DE_CODIGO.md](#), Parte 2. Insertarlo en el Capítulo 4, sección de diseño de la base de datos.

A.5 Diagramas de Casos de Uso

Diagramas de Casos de Uso — Ecolingo



Dos actores — Estudiante y Docente — sin superposición de casos de uso salvo el reseteo de PIN, que conecta a ambos (el docente lo ejecuta, el estudiante lo origina al pedirlo). Los casos con borde punteado ocurren fuera de la plataforma (Google Forms, comunicación verbal del PIN). Insertarlo en el Capítulo 4, sección de requisitos o de diseño funcional — donde el tutor lo haya pedido en su informe de correcciones.

A.6 Scaffold del frontend

Responsable: Persona A. **Duración:** 45 min.

Se instalan y configuran las dependencias, los tokens de diseño y el enrutador. **No se construye ninguna pantalla.**

3.1 Dependencias

```
cd frontend
npm install
npm install react-router-dom axios framer-motion recharts
npm install -D tailwindcss@3 postcss autoprefixer
npx tailwindcss init -p
```

Paquete	Para qué	Origen
react-router-dom	Navegación entre pantallas	Diseño p. 12

Paquete	Para qué	Origen
<code>axios</code>	Llamadas al backend	GUIA_DE_CODIGO.md §0.3
<code>framer-motion</code>	Confetti y transiciones	Diseño p. 12
<code>recharts</code>	Gráficos del panel del docente	Diseño p. 12 · decisión T-03
<code>tailwindcss</code> v3	Estilos	Diseño p. 12

Tailwind v3, no v4. El Documento de Diseño especifica v3, y v4 cambió el sistema de configuración: ya no usa `tailwind.config.js` de la misma forma. Instalar v4 rompería los tokens tal como están definidos. Fijá la versión explícitamente.

3.2 Tokens de diseño

`frontend/tailwind.config.js` — paleta exacta del Documento de Diseño, p. 2.

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        bg: '#0d1117', // fondo de la aplicación
        surface: '#161b22', // cards, módulos, filas de ranking
        primary: '#1a6b3a', // botones principales, bordes activos
        accent: '#4ade80', // XP, destacados, respuestas correctas
        info: '#60a5fa', // módulo activo, posición propia en ranking
        warn: '#f59e0b', // racha diaria, estadísticas de posición
        danger: '#ef4444', // respuestas incorrectas, vida perdida
        text: '#e6f5ee', // títulos y texto principal
        muted: '#8b949e', // descripciones, subtítulos, etiquetas
      },
      borderWidth: { hair: '0.5px' },
    },
  },
  plugins: [],
};
```

`frontend/src/index.css`

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html { -webkit-text-size-adjust: 100%; }
  body {
    @apply bg-bg text-text antialiased;
    font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
```

```

}
/* Sin sombras: la profundidad se logra con capas de color (Diseño p. 12)
*/
* { box-shadow: none; }
}

```

Regla que evita el 80 % de la inconsistencia visual: ningún componente escribe un color hexadecimal. Siempre `bg-surface`, `text-accent`, `border-primary`. Si hace falta un color que no está en la paleta, se agrega acá primero y se avisa al otro. Está en la definición de terminado de `GUIA_DE_CODIGO.md §0.1`.

3.3 Cliente de API

`frontend/src/api/client.js` — implementa `GUIA_DE_CODIGO.md §0.3. Zona compartida`.

```

import axios from 'axios';

const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' },
});

client.interceptors.request.use((config) => {
  const token = localStorage.getItem('ecolingo_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

client.interceptors.response.use(
  (res) => res,
  (err) => {
    const code = err.response?.data?.error?.code;
    if (code === 'EXPIRED_TOKEN' || code === 'INVALID_TOKEN' || code === 'MISSING_TOKEN') {
      localStorage.removeItem('ecolingo_token');
      if (!window.location.pathname.startsWith('/login')) {
        window.location.assign('/login');
      }
    }
    return Promise.reject(err);
  }
);

/** Extrae un mensaje mostrable de cualquier error de la API. */
export const getErrorMessage = (err) =>
  err.response?.data?.error?.message ??
  'No pudimos conectar con el servidor. Revisá tu conexión e intentá de nuevo.';

```

```
export default client;
```

timeout: 15000 y no los 10 s del contrato: el arranque en frío de Render puede tardar más de 10 segundos y un timeout corto haría fallar la primera petición del día con un error de red engañoso.

3.4 Entorno del frontend

frontend/.env (no se commitea)

```
VITE_API_URL=http://localhost:4000/api/v1
```

frontend/.env.example (sí se commitea)

```
VITE_API_URL=http://localhost:4000/api/v1
```

En Vite, **solo** las variables con prefijo **VITE_** llegan al navegador. Es también la advertencia: todo lo que tenga ese prefijo es público. Nunca pongas ahí un secreto.

3.5 Enrutador mínimo

frontend/src/App.jsx — solo el esqueleto. Las pantallas llegan con F02.

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';

const Placeholder = ({ name }) => (
  <div className="min-h-screen flex items-center justify-center">
    <p className="text-muted">Pantalla pendiente: {name}</p>
  </div>
);

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login"      element={<Placeholder name="Login" />} />
        <Route path="/mapa"      element={<Placeholder name="Mapa de
Módulos" />} />
        <Route path="*"          element={<Navigate to="/login" replace />}
      />
      </Routes>
    </BrowserRouter>
  );
}
```

Las rutas definitivas están en GUIA_DE_CODIGO.md §0.12. Se completan en F02.

3.6 Verificar

```
npm run dev
```

Abrió <http://localhost:5173>. Deberías ver el texto sobre **fondo oscuro**. Si el fondo es blanco, Tailwind no está tomando la configuración: revisá que `index.css` esté importado en `main.jsx` y que `content` en `tailwind.config.js` apunte a `./src/**/*.{js,jsx}`.

✓ Checkpoint

- ☐ `npm run dev` levanta en `localhost:5173`
- ☐ El fondo es `#0d1117` — Tailwind está activo
- ☐ `client.js` creado con ambos interceptores
- ☐ `.env` ignorado, `.env.example` commiteado

A.7 Despliegue vacío de verificación

Responsable: ambos, en pareja. **Duración:** 45 min.

El backend en Render se despliega siguiendo `GUIA_DE_CODIGO.md §0.7` (conexión y esqueleto) — las variables de entorno de producción están en el Bloque 1.2 de la Parte B de este mismo documento. Desplégalo primero y volvé acá para el frontend.

4.1 Frontend en Vercel

En <https://vercel.com/new>, importá el repositorio:

Campo	Valor
Framework Preset	Vite
Root Directory	<code>frontend</code>
Build Command	<code>npm run build</code> (por defecto)
Output Directory	<code>dist</code> (por defecto)
Install Command	<code>npm install</code> (por defecto)

Variables de entorno:

Clave	Valor	Entornos
<code>VITE_API_URL</code>	<code>https://<tu-servicio>.onrender.com/api/v1</code>	Production, Preview

Las variables de Vite se resuelven en tiempo de build, no de ejecución. Si cambiás `VITE_API_URL` después, hay que redespargar — no alcanza con guardar. Es una fuente clásica de confusión.

4.2 Rewrite para el enrutador

React Router maneja las rutas en el cliente. Sin esta configuración, entrar directo a <https://ecolingo.vercel.app/mapa> devuelve 404, porque Vercel busca un archivo físico en esa ruta.

frontend/vercel.json

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Sin esto, la app funciona al navegar pero se rompe al refrescar la página o al compartir un enlace. Es exactamente el tipo de fallo que aparece recién en el aula.

4.3 CORS

Con la URL de Vercel ya asignada, actualizá en Render:

Clave	Valor
CORS_ORIGIN	https://ecolingo.vercel.app

Si querés que funcionen también las previews de las ramas, separá por coma. El backend ya hace `split(',')` (GUIA_DE_CODIGO.md §0.7).

4.4 Despliegues automáticos

Rama	Vercel	Render
main	Producción	Producción
dev	Preview con URL fija	(opcional)
feat/*	Preview por PR	—

Las previews de Vercel por PR son gratis y muy útiles: cada uno puede ver la pantalla del otro sin bajarse la rama.

✅ Checkpoint

- ☐ Backend desplegado, `/api/v1/health` responde `database: "connected"`
- ☐ Frontend desplegado, carga con fondo oscuro
- ☐ `vercel.json` con el rewrite
- ☐ `CORS_ORIGIN` apunta a la URL real de Vercel
- ☐ Ambos con acceso a los dos paneles

Bloque 5 — Verificar la cadena completa

Responsable: ambos. **Duración:** 20 min.

Esto es lo que valida todo el bloque anterior: **el frontend desplegado hablando con el backend desplegado hablando con Atlas.**

5.1 Ping temporal

Agregá esto a `App.jsx`, provisorio:

```
import { useEffect, useState } from 'react';
import client from './api/client';

function HealthCheck() {
  const [state, setState] = useState({ status: 'cargando' });

  useEffect(() => {
    client.get('/health')
      .then((r) => setState(r.data))
      .catch((e) => setState({ status: 'error', message: e.message }));
  }, []);

  return (
    <div className="min-h-screen flex items-center justify-center p-6">
      <pre className="bg-surface border-hair border-primary rounded-lg p-6 text-accent text-sm">
        {JSON.stringify(state, null, 2)}
      </pre>
    </div>
  );
}
```

Montalo en / temporalmente y desplegó.

5.2 Qué tiene que pasar

En la URL de Vercel, sobre fondo oscuro:

```
{
  "status": "ok",
  "environment": "production",
  "database": "connected",
  "timestamp": "2026-08-08T..."
}
```

Si ves eso, las tres capas están conectadas en producción. Es el hito del pre-código.

Puede tardar hasta un minuto la primera vez — es el arranque en frío de Render (Bloque 6 de la Parte B de este documento). Refrescá.

5.3 Si falla

Síntoma en la consola del navegador	Causa	Solución
<code>blocked by CORS policy</code>	<code>CORS_ORIGIN</code> no coincide con la URL de Vercel	Copiala exacta, sin barra final. Redesplegá Render.
<code>Network Error</code> sin más detalle	El backend está dormido, o <code>VITE_API_URL</code> está mal	Abrí <code>/api/v1/health</code> directo en el navegador. Si responde, el problema es la variable de Vite — recordá redesplegar.
Responde pero <code>database: "disconnected"</code>	<code>MONGODB_URI</code> mal cargada en Render	Revisá el Bloque 1.2 de la Parte B de este documento.
<code>404</code> al refrescar una ruta	Falta el rewrite	<code>vercel.json</code> (§4.2).
Funciona en local pero no en producción	Variable de Vite cargada después del build	Redesplegá desde el panel de Vercel.

5.4 Limpiar

Quitá el `HealthCheck` y restaurá el enrutador del §3.5. **No lo dejes en `main`.**

```
git checkout -b chore/verificar-despliegue
git commit -am "chore(deploy): verificar cadena completa en producción"
```

✓ Checkpoint

- ☐ La URL de Vercel muestra `status: ok` y `database: connected` desde producción
- ☐ Ambos lo vieron funcionando
- ☐ El código de prueba fue removido

A.8 Checklist de cierre de la Parte A

Con esto cerrado, **arranca el trabajo de features.**

Cuentas

- ☐ GitHub, Atlas, Render y Vercel activos
- ☐ Ambos con permisos de administrador en los cuatro
- ☐ Canal privado de credenciales acordado

Repositorio

- ☐ Monorepo `frontend/` + `backend/` + `docs/`

- ☐ `main` y `dev` protegidas, sin auto-merge
- ☐ Plantillas de PR e Issue
- ☐ Etiquetas `F01–F12` y auxiliares
- ☐ Los 3 documentos (Tesis, `GUIA_DE_CODIGO.md`, `GUIA_DEL_PROYECTO.md`) en `docs/`
- ☐ Ningún secreto en el historial

Local

- ☐ Ambos levantan frontend y backend en su máquina
- ☐ Ambos con su `.env` a partir de los `.env.example`
- ☐ Tokens de Tailwind aplicados
- ☐ `client.js` con interceptores

Producción

- ☐ Backend en Render con `/health` en `connected`
- ☐ Frontend en Vercel con fondo oscuro
- ☐ `vercel.json` con rewrite
- ☐ CORS correcto
- ☐ Cadena completa verificada desde la URL pública

Decisiones

- ☐ Quién es Persona A y quién Persona B
- ☒ Política de `studentCode` — A: código asignado por el docente

Documentación

- ☐ `GUIA_DE_CODIGO.md` y este documento re-subidos al contexto de Claude de ambos, si algo cambió en esta fase

FIN DE LA PARTE A — no continúes sin esto cerrado

No sigas a la Parte B hasta que:

- ☐ Todo el checklist de la Parte A (arriba) esté cerrado
- ☐ Las features del núcleo de `GUIA_DE_CODIGO.md` (F01 a F05, F09, F11 — ver el mapa de features, Parte 0.10) estén implementadas y probadas en local contra `ecolingo_dev`
- ☐ Los tres diagramas de este documento ya están insertados en el Capítulo 4 de la tesis

La Parte B es sobre software que ya existe. Antes de eso no hay nada que desplegar.

PARTE B — Después del código

Objetivo y alcance

Esta guía cubre lo que pasa **después** de que las features estén hechas: el pase a producción, la operación durante las cuatro semanas de piloto, y el cierre con los datos asegurados.

Qué está en otros documentos

Tema	Dónde
Crear los servicios en Render y Vercel	este documento, Parte A.7
Variables de entorno del backend	<code>GUIA_DE_CODIGO.md</code> §0.5 (variables de entorno)
Sembrar e indexar <code>ecolingo_prod</code>	<code>GUIA_DE_CODIGO.md</code> §0.11 (seeds) — ejecutado contra producción con <code>CONFIRM_PROD_SEED=yes</code>
Script de respaldo	el script de respaldo de esta Parte B, Bloque 9.1

Acá está lo que ninguno cubre: **el procedimiento de release, la verificación, el rollback, el monitoreo y qué hacer cuando algo se rompe en medio de una clase.**

Una idea que conviene tener presente

El monitoreo de este proyecto tiene doble propósito. Es operación —que la plataforma no se caiga— pero también es **recolección de evidencia para el Capítulo 5**. La tesis §4.5 compromete tiempos de respuesta, disponibilidad y capacidad concurrente. Sin registros, esas afirmaciones no se pueden sostener ante el tribunal. El Anexo de este documento indica qué guardar.

Bloque 1 — Entornos

1.1 Separación

Entorno	Frontend	Backend	Base de datos	Rama
Local	<code>localhost:5173</code>	<code>localhost:4000</code>	<code>ecolingo_dev</code>	cualquiera
Preview	URL efímera de Vercel	Render producción	<code>ecolingo_dev</code>	<code>feat/*</code> , <code>dev</code>

Entorno	Frontend	Backend	Base de datos	Rama
Producción	dominio de Vercel	Render producción	ecolingo_prod	main

El punto crítico es que un mismo backend sirve a preview y a producción, apuntando a bases distintas según la variable de entorno. Con el plan gratuito de Render no se puede tener dos servicios, así que la separación real la da **MONGODB_URI**.

Consecuencia: mientras el piloto está en curso, el backend de Render apunta a **ecolingo_prod**. Cualquier prueba desde una preview de Vercel escribe en la base del piloto. **Durante las cuatro semanas, probá solo en local.**

1.2 Variables por entorno

Clave	Local	Producción
NODE_ENV	development	production
MONGODB_URI	.../ecolingo_dev	.../ecolingo_prod
CORS_ORIGIN	http://localhost:5173	dominio de Vercel
JWT_SECRET	cualquiera	distinto al de desarrollo
VITE_API_URL	http://localhost:4000/api/v1	https://...onrender.com/api/v1

Que el **JWT_SECRET** de producción sea distinto no es formalismo: si se filtra el de desarrollo —que circula entre los dos y aparece en más lugares— las sesiones del piloto siguen protegidas.

1.3 Congelamiento de producción

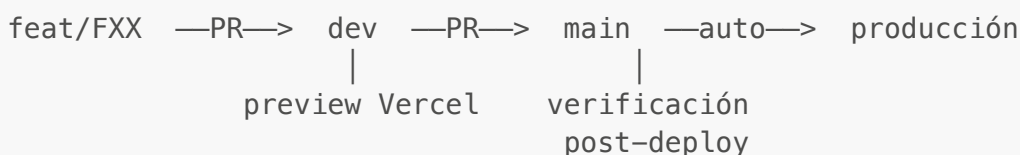
Desde el día en que el primer alumno entra:

Prohibido	Motivo
Correr seeds sobre ecolingo_prod	Sobrescribe contenido en uso
deleteOne o deleteMany en producción	Los datos del piloto no se regeneran
Editar documentos desde el Data Explorer	Se saltea toda la validación de Mongoose
Cambiar esquemas sin respaldo previo	—
Mergear a main sin haber probado en local	—

Bloque 2 — Procedimiento de release

Responsable: el primero, en pareja. Los siguientes, quien tenga el cambio. **Duración:** 20–30 min.

2.1 Flujo



Nunca se mergea directo a **main**. Todo pasa por **dev** primero.

2.2 Antes de mergear a **main**

- ☐ Todas las features del release probadas en local contra **ecolingo_dev**
- ☐ **dev** desplegada en una preview y revisada por ambos
- ☐ Sin **console.log** de depuración en el diff
- ☐ Sin credenciales ni **.env** en el diff
- ☐ - [] **Respaldo de ecolingo_prod hecho** si el release toca esquemas
- ☐ Ningún alumno usando la plataforma en este momento

Nunca despliegues durante una clase. Render reinicia el contenedor al desplegar: entre 30 y 60 segundos de caída. Si eso pasa mientras 20 alumnos están respondiendo, se pierden intentos en curso y la sesión se arruina. Desplegá a la noche o en días sin clase.

2.3 El release

```

git checkout dev && git pull
git checkout main && git pull
git merge dev --no-ff -m "release: <features incluidas>"
git tag -a v0.X.0 -m "Release: <descripción>"
git push origin main --tags
  
```

--no-ff fuerza un commit de merge, así el historial muestra cada release como un punto identificable. Las etiquetas permiten volver a una versión exacta si hay que hacer rollback.

Render y Vercel despliegan solos al detectar el push. Render tarda 2–5 minutos; Vercel, 1–2.

2.4 Si el release toca la base

Cambio	Qué hacer antes
Campo nuevo en un esquema	Respaldo + script de actualización de documentos existentes
Índice nuevo	Respaldo + node src/scripts/createIndexes.js contra producción
Contenido nuevo (preguntas, glosario)	Respaldo + seed con CONFIRM_PROD_SEED=yes
Solo código	Nada

Bloque 3 — Verificación post-despliegue

Obligatoria después de cada release. 10 minutos.

3.1 Automática

```
curl -s https://<tu-servicio>.onrender.com/api/v1/health | jq
```

```
{ "status": "ok", "environment": "production", "database": "connected",  
  "timestamp": "..."} }
```

Si **environment** no dice **production**, la variable no se cargó. Si **database** no dice **connected**, revisá **MONGODB_URI** en Render.

3.2 Manual — recorrido completo

Desde el dominio real, **en un celular**, con una cuenta de prueba en la clase de pruebas:

#	Acción	Esperado
1	Entrar como alumno	Llega al mapa
2	Ver el mapa	Módulo 1 disponible, resto bloqueado
3	Iniciar el módulo 1	Carga la primera situación
4	Responder bien	+50 XP, explicación visible
5	Responder mal	Vida menos, explicación visible
6	Terminar el módulo	Pantalla de resultado con XP e insignia
7	Volver al mapa	Módulo 2 desbloqueado, XP actualizado
8	Abrir Ranking	Aparece la posición propia
9	Abrir Logros	Insignia nueva a color
10	Buscar un término	Devuelve resultados
11	Abrir Perfil	Datos correctos
12	Refrescar la página en /mapa	Carga bien, no da 404
13	Cerrar sesión y volver a entrar	Progreso conservado
14	Entrar como docente	Panel con datos reales
15	Exportar CSV	Descarga y abre bien en Excel con tildes

El paso 12 es el que detecta que falta el rewrite de `vercel.json`. Es un fallo que no aparece navegando, solo al refrescar o al abrir un enlace compartido — o sea, aparece recién en el aula.

Usá una clase de pruebas separada (TEST2026), nunca la del piloto.

3.3 Si algo falla

Rollback inmediato (Bloque 4), arreglar en `dev`, volver a desplegar. **No dejes producción rota mientras investigás.**

Bloque 4 — Rollback

Dos vías. La primera es más rápida.

4.1 Desde los paneles — 2 minutos

Vercel: Deployments → elegir el despliegue anterior que funcionaba → *Promote to Production*. Es instantáneo.

Render: Events → elegir el despliegue anterior → *Rollback*. Tarda 2–3 minutos.

Esto revierte lo desplegado sin tocar el repositorio. Es lo que hay que hacer si algo se rompe con alumnos conectados.

4.2 Desde Git — cuando hay tiempo

```
git checkout main
git revert -m 1 <hash-del-merge>
git push origin main
```

`revert` crea un commit nuevo que deshace el anterior. **No uses `reset` ni `push --force` sobre `main`:** reescribe historial en una rama protegida y compartida.

4.3 Rollback de datos

Si un release corrompió datos, revertir el código no alcanza. Hay que restaurar desde el último respaldo (el script de respaldo de esta Parte B, Bloque 9.1) — y por eso el respaldo previo a cualquier cambio de esquema no es opcional.

Bloque 5 — Pruebas antes del piloto

Cuándo: la semana anterior al primer uso con alumnos. **Responsable:** ambos. **Duración:** 3–4 h.

5.1 Prueba con personas ajenas

Conseguí **2 o 3 personas que no sean ustedes** — hermanos, compañeros, cualquiera. Idealmente de edad parecida a los alumnos.

Dales solo el enlace y el código de clase. **No expliques nada**. Observá en silencio y anotá:

- ¿Entienden qué hacer en la pantalla de login sin ayuda?
- ¿Encuentran cómo empezar un módulo?
- ¿Se dan cuenta de que perdieron una vida?
- ¿Leen la explicación o la saltan?
- ¿Dónde dudan?

Todo lo que tengas que explicarles es un problema de usabilidad. La tesis compromete una interfaz *"intuitiva y accesible para usuarios de 14 a 18 años sin experiencia técnica previa"* (§4.5): esta prueba es la evidencia de que se verificó, y sirve directo para el Capítulo 5.

5.2 Prueba en dispositivos reales

Dispositivo	Qué mirar
Android de gama baja	Rendimiento de las animaciones, renderizado de emojis
iPhone	Safari se comporta distinto de Chrome
Pantalla de 375 px	Que nada se corte ni se solape
Computadora del aula	Si es donde van a usarla

Verificá que los emojis de las insignias se vean. En Android viejos pueden salir como cuadrados vacíos. Si pasa, hay que reemplazarlos por iconos SVG.

5.3 Prueba de carga

La tesis compromete **30 usuarios simultáneos sin degradación** (§4.5). Vale la pena medirlo para poder afirmarlo con datos.

```
npx autocannon -c 30 -d 30 https://<tu-servicio>.onrender.com/api/v1/health
```

30 conexiones concurrentes durante 30 segundos. Anotá latencia media, percentil 99 y errores.

Para algo más realista, probá también un endpoint autenticado con un token válido. Guardá la salida: es la evidencia del RNF de rendimiento.

Si la latencia se dispara, es esperable en un plan compartido gratuito. Lo importante es **medirlo y reportarlo honestamente**, no que el número sea perfecto.

5.4 Ensayo general

Una semana antes, con el docente:

- ☐ El docente entra a su panel y navega solo

- ☐ Sabe habilitar el pre-test
- ☐ Sabe exportar el CSV
- ☐ Tiene la lista de códigos de alumno lista para repartir
- ☐ Probaron el wifi del aula con varios dispositivos a la vez
- ☐ Acordaron qué hacer si la plataforma no carga

Bloque 6 — Arranque en frío

Render suspende los servicios gratuitos tras 15 minutos sin tráfico. La primera petición después tarda **30 a 60 segundos**.

Esto choca de frente con el RNF de la tesis §4.5: *"responder en un tiempo inferior a tres segundos"*.

6.1 Las opciones

Opción	Cómo	Costo	Efectividad
Precalentar antes de clase	Abrir la app 5 min antes	Gratis	Alta, si no se olvidan
Ping externo	UptimeRobot cada 10 min a <a href="https://<tu-servicio>.onrender.com/api/v1/health">/api/v1/health	Gratis	Alta
Plan pago	Plan más barato de Render	~7 USD/mes	Total

6.2 Recomendación

Combiná las tres, en este orden: configurá el *ping* externo esta semana —es gratis y resuelve el 90 % del problema—, mantené el ritual de precalentar antes de cada clase, y considerá pagar el plan durante las cuatro semanas del piloto. Siete dólares por mes para proteger el instrumento de medición de un TCC es de las mejores relaciones costo-beneficio del proyecto.

6.3 Configurar el ping

En UptimeRobot (u otro monitor gratuito):

Campo	Valor
Tipo	HTTP(s)
URL	<a href="https://<tu-servicio>.onrender.com/api/v1/health">https://<tu-servicio>.onrender.com/api/v1/health
Intervalo	10 minutos
Alerta	Email de ambos

Además de mantener el servicio despierto, esto te da **un registro de disponibilidad** — que es exactamente la evidencia del RNF de 95 % que promete la tesis.

6.4 Qué decir en la defensa

Si preguntan por tiempos de respuesta, la respuesta correcta es:

La latencia observada en el primer acceso tras un período de inactividad corresponde a la suspensión de instancias del plan gratuito de la plataforma de despliegue, no al diseño del sistema. En uso sostenido, la respuesta se mantiene por debajo del umbral comprometido. Se mitigó con un monitor externo que mantiene la instancia activa.

Es honesto, técnicamente correcto y demuestra que entendieron la infraestructura.

Bloque 7 — Monitoreo durante el piloto

Duración: 4 semanas, mediados de septiembre a mediados de octubre.

7.1 Guardia rotativa

Cada semana, uno de los dos es el **responsable de guardia**: revisa los indicadores, atiende incidentes y hace el respaldo del viernes.

Semana	Guardia
1	Persona A
2	Persona B
3	Persona A
4	Persona B

El otro sigue desarrollando las features extra. Sin esto, o los dos miran todo el tiempo o ninguno mira.

7.2 Rutina diaria — 5 minutos

Días con clase:

- ☐ `/api/v1/health` responde `connected`
- ☐ UptimeRobot sin caídas en las últimas 24 h
- ☐ Render → Logs: sin errores 500
- ☐ Panel del docente: los alumnos activos coinciden con lo esperado

7.3 Rutina semanal — 20 minutos, viernes

- ☐ **Respaldo de `ecolingo_prod`** (el script de respaldo de esta Parte B, Bloque 9.1)
- ☐ Atlas → Metrics: tamaño de la base y conexiones
- ☐ Render → Metrics: memoria y latencia
- ☐ Exportar el CSV de la semana y guardarlo
- ☐ Revisar el registro de incidentes
- ☐ Avisar al otro cómo viene

7.4 Qué mirar y cuándo preocuparse

Indicador	Dónde	Normal	Alerta
Disponibilidad	UptimeRobot	> 99 %	< 95 %
Errores 500	Logs de Render	0	cualquiera
Tamaño de la base	Atlas Metrics	< 5 MB	> 400 MB
Conexiones	Atlas Metrics	< 20	> 400
Memoria	Render Metrics	estable	crecimiento sostenido
Alumnos activos	Panel del docente	según lo acordado	caída brusca

Un crecimiento sostenido de memoria en Render suele indicar una fuga, casi siempre por conexiones a Mongo que no se cierran o por listeners que se acumulan. Con un piloto de cuatro semanas probablemente no llegue a ser un problema, pero conviene anotarlo si aparece.

7.5 Registro de incidentes

Llevá un archivo `docs/INCIDENTES.md`. Cada evento:

```
## 2026-09-23 – Lentitud al inicio de la clase
```

```

**Detectado por:** el docente, al abrir la plataforma
**Impacto:** ~45 s de espera para el primer alumno
**Causa:** arranque en frío de Render; el ping estaba mal configurado
**Solución:** se corrigió el intervalo del monitor a 10 min
**Prevención:** precalentar 5 min antes, agregado a la rutina del docente
**Tiempo hasta resolución:** 10 min

```

Esto no es burocracia: **es material del Capítulo 5**. Un TCC que documenta sus incidentes operativos y cómo los resolvió es notablemente más sólido que uno que afirma que todo salió perfecto.

Bloque 8 — Runbook de incidentes

Para consultar **durante** un problema, no después. Guardalo accesible desde el celular.

8.1 "La plataforma no carga" — en medio de una clase

1. Abrí `/api/v1/health` en el celular
 - └ Responde ok → el backend está bien, seguí al paso 2
 - └ Tarda pero responde → arranque en frío. Avisá: "un minuto y arranca"
 - └ No responde → paso 3

2. Abrí el dominio de Vercel
 - ├ Carga → problema del wifi del aula o de un dispositivo
 - └ No carga → Vercel → Deployments → Promote al anterior
3. Render → Events
 - ├ Deploy fallido → Rollback al anterior (2–3 min)
 - ├ Servicio suspendido → Resume
 - └ Todo normal → paso 4
4. Atlas → clúster activo?
 - ├ No → esperá; suele resolverse solo
 - └ Sí → revisá Network Access: ¿sigue 0.0.0.0/0?

Mientras tanto, decile al docente que siga con la clase sin la plataforma. Perder 20 minutos de una sesión no arruina el piloto; que el docente se quede parado frente al curso, sí.

8.2 Tabla de síntomas

Síntoma	Causa probable	Acción
Todo lento la primera vez del día	Arranque en frío	Esperar. Revisar el ping.
Error de CORS	CORS_ORIGIN no coincide	Corregir en Render y redespargar
404 al refrescar una ruta	Falta el rewrite	vercel.json
"Sesión expirada" constante	JWT_SECRET cambió	Restaurar el valor anterior
Un alumno no puede entrar	Código mal tipeado, o STUDENT_CODE_TAKEN	Verificar en el panel del docente
El XP no se suma	Repetición de módulo	Es el comportamiento correcto (D-08)
El ranking no cambia	Cache o corte semanal	Refrescar; verificar weeklyXpResetAt
CSV con caracteres raros	Falta el BOM	Corregir la exportación
500 en un endpoint	Error no controlado	Logs de Render → identificar → arreglar en dev

8.3 Cuándo hacer rollback y cuándo no

Rollback inmediato: los alumnos no pueden entrar, se pierden datos, o un endpoint del núcleo devuelve 500.

Arreglar sin rollback: un problema visual, algo que afecta a un solo alumno, o una feature extra que falla.

En duda, hacé rollback. Volver a una versión que funcionaba es siempre más barato que depurar en producción con alumnos conectados.

Bloque 9 — Mantenimiento

9.1 Respaldos

El calendario está en la el calendario de la Bloque 9.1 de esta misma parte. Recordatorio de lo esencial:

Momento	Acción
Antes del piloto	Respaldo de <code>ecolingo_prod</code> sembrada
Cada viernes	Respaldo semanal — lo hace la guardia
Antes de cada release con cambio de esquema	Respaldo previo
Al cerrar el piloto	Respaldo definitivo, en dos lugares distintos

9.2 Secretos

Situación	Acción
Un secreto llegó a GitHub	Rotarlo de inmediato. Borrar el commit no alcanza: queda en el historial.
Fin del piloto	Rotar <code>JWT_SECRET</code> y la contraseña de <code>ecolingo_app</code>

9.3 Límites de los planes gratuitos

Servicio	Límite	Margen
Atlas	512 MB, 500 conexiones	El piloto usa ~1,5 MB. Sin riesgo.
Render	Suspensión por inactividad, 750 h/mes	750 h alcanza para un servicio siempre activo
Vercel	100 GB de ancho de banda	Muy por encima de lo necesario

Ninguno es una limitación real para este proyecto, salvo la suspensión de Render.

9.4 Qué NO hacer durante el piloto

- Actualizar dependencias
- Refactorizar
- Cambiar esquemas sin necesidad real
- Desplegar en día de clase

Las cuatro semanas del piloto son de **congelamiento de funcionalidad**. Solo entran correcciones de errores que impidan usar la plataforma. Toda mejora espera. La tentación de "aprovechar y

arreglar esto" es la principal fuente de incidentes evitables.

Bloque 10 — Cierre del piloto

Cuándo: mediados de octubre, al terminar la cuarta semana. **Responsable:** ambos.

10.1 Secuencia

- ☐ Confirmar con el docente que terminó el uso
- ☐ Verificar que el post-test se aplicó a todos
- ☐ **Respaldo definitivo de `ecolingo_prod`**
- ☐ Guardarlo en **dos lugares**: Drive y un disco local
- ☐ Exportar los tres CSV: `summary`, `attempts`, `answers`
- ☐ Exportar las respuestas de Google Forms a CSV
- ☐ Descargar el informe de disponibilidad de UptimeRobot
- ☐ Guardar los resultados de la prueba de carga
- ☐ Cerrar `docs/INCIDENTES.md`

10.2 Mantener la plataforma en línea

No la apagues. Dejela funcionando hasta después de la defensa:

- El tribunal puede pedir una demostración en vivo
- Puede hacer falta volver a exportar datos
- Apagarla y volver a levantarla es un riesgo innecesario

Sí conviene: poner las clases en `isActive: false` para que no entren alumnos nuevos, y mantener el monitor activo.

10.3 Antes de la defensa

- ☐ La plataforma responde
- ☐ Hay una clase de demostración con datos representativos
- ☐ El recorrido completo funciona en el dispositivo que van a usar
- ☐ Los respaldos siguen accesibles
- ☐ Tienen a mano las respuestas sobre arranque en frío, seguridad de la base y datos de menores

Checklists

Antes del primer despliegue a producción

- ☐ Todas las features del núcleo terminadas y aprobadas
- ☐ `ecolingo_prod` sembrada e indexada
- ☐ Variables de producción cargadas en Render y Vercel
- ☐ CORS apuntando al dominio real
- ☐ `vercel.json` con el rewrite

- ☐ Recorrido completo verificado en producción, desde un celular
- ☐ Monitor externo configurado
- ☐ Respaldo inicial hecho
- ☐ Runbook accesible desde el celular de ambos

Antes de cada clase

- ☐ Precalentar la app 5 minutos antes
- ☐ Verificar **/health**
- ☐ Confirmar que no hay despliegue en curso

Cada viernes

- ☐ Respaldo semanal
- ☐ Revisar métricas de Atlas y Render
- ☐ Exportar CSV de la semana
- ☐ Actualizar el registro de incidentes
- ☐ Traspasar la guardia

Al cerrar

- ☐ Respaldo definitivo en dos lugares
- ☐ Los tres CSV exportados
- ☐ Informe de disponibilidad guardado
- ☐ Registro de incidentes cerrado
- ☐ Plataforma en línea, clases inactivas

Anexo — Evidencia para el Capítulo 5

Lo que hay que guardar durante el piloto para poder sostener las afirmaciones de la tesis.

Afirmación de la tesis	Evidencia	De dónde sale
Respuesta < 3 s (§4.5)	Prueba de carga + métricas de Render	Bloque 5.3, Render Metrics
30 usuarios simultáneos (§4.5)	Salida de autocannon	Bloque 5.3
Disponibilidad ≥ 95 % (§4.5)	Informe de UptimeRobot	Bloque 6.3
Interfaz intuitiva para 14–18 años (§4.5)	Observaciones de la prueba con personas ajenas + SUS	Bloque 5.1
Responsiva en tres tamaños (§4.5)	Capturas en 375, 768 y 1280 px	Bloque 5.2
Máximo tres clics (§4.5)	Mapa de navegación verificado	F02

Afirmación de la tesis	Evidencia	De dónde sale
Contraseñas con bcrypt, JWT, HTTPS (§4.5)	Código + configuración	GUIA_DE_CODIGO.md §0.2 (JWT) y §0.5 (variables de entorno)
Ley 6.529/2020, datos de menores (§4.5)	las decisiones D-03 y D-04 de GUIA_DE_CODIGO.md	el registro de decisiones de GUIA_DE_CODIGO.md
Registro de uso y engagement	moduleAttempts y CSV answers	Panel del docente
Mejora de retención ≥ 25 %	Pre/post-test de Google Forms	—

Guardá todo en una carpeta **evidencia/ fuera del repositorio,** junto con los respaldos. Contiene datos de alumnos.

El registro de incidentes merece una mención aparte. Un capítulo de resultados que describe qué falló, por qué y cómo se resolvió es más convincente que uno que afirma que todo funcionó sin problemas. Los tribunales reconocen la diferencia entre un proyecto que se operó de verdad y uno que se demostró una vez.

Historial de versiones

Versión	Fecha	Cambios
1.0	Agosto 2026	Versión inicial. 10 bloques, runbook de incidentes, anexo de evidencia.

Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026

Anexo B — Historial de versiones

Versión	Fecha	Cambios
1.0	Agosto 2026	Reemplaza a 01-PRE-CODIGO.md y 03-DESPLIEGUE-Y-MONITOREO.md. Une ambos en un solo documento con divisor Parte A / Parte B. Agrega los tres diagramas del TCC (arquitectura, esquema de datos, casos de uso) a la Parte A. Elimina las referencias a 00-INDICE.md, que no existe en el sistema de 3 archivos.

Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026